Reinforcement Learning

Chapter 5: RL via Policy Gradient

Ali Bereyhi

ali.bereyhi@utoronto.ca

Department of Electrical and Computer Engineering

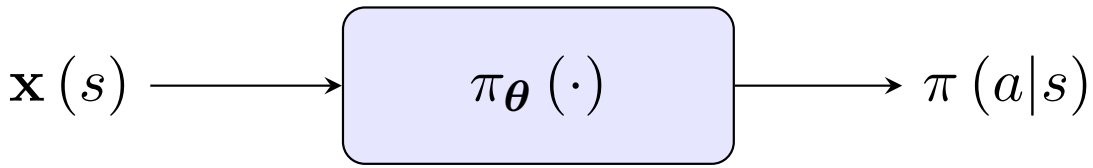
University of Toronto

Fall 2026

Policy Network

Policy Network

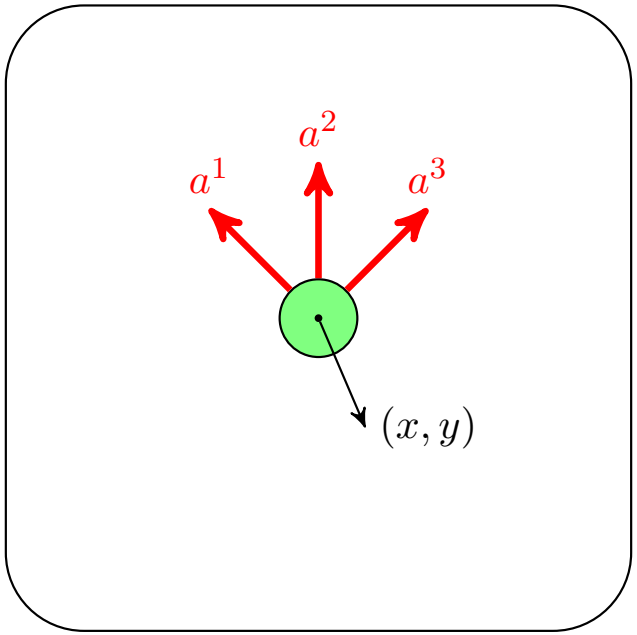
(Practical) policy networks are approximation models that map state features to a conditional probability distribution on actions



+ *How can we realize such a network? It should return distribution!*
– Yes! That's right! Let's see a few examples

Example: *Moving Particle*

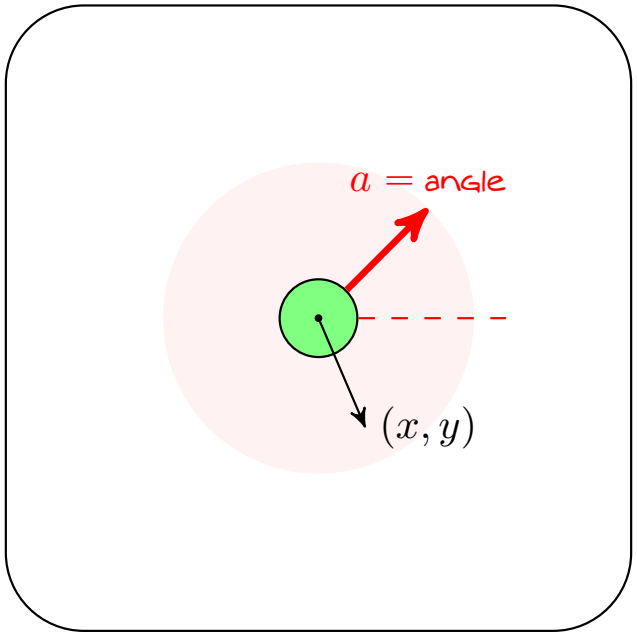
We are controlling a moving particle that could move in the 2D space



$$\mathbf{x}(s) = \begin{bmatrix} x \\ y \end{bmatrix} \rightsquigarrow \pi_{\boldsymbol{\theta}}(\mathbf{a}|s) = \begin{bmatrix} \pi_{\boldsymbol{\theta}}(a^1|s) \\ \pi_{\boldsymbol{\theta}}(a^2|s) \\ \pi_{\boldsymbol{\theta}}(a^3|s) \end{bmatrix}$$

Example: *Moving Particle*

We have the same moving particle that could move in any direction



$$\mathbf{x}(s) = \begin{bmatrix} x \\ y \end{bmatrix} \rightsquigarrow \pi_{\boldsymbol{\theta}}(a|s) \equiv \mathcal{N}\left(\mu_{\boldsymbol{\theta}}(s), \sigma^2_{\boldsymbol{\theta}}(s)\right)$$

New Notation

New Notation

As we think of a generic action and state space, we use a simple notation

$$\int_a f(a) = \begin{cases} \sum_{a \in \mathbb{A}} f(a) & \text{discrete } a \\ \int_{\mathbb{A}} f(a)\,\mathrm{d}a & \text{continuous } a \end{cases}$$

Example: *Softmax*

The most basic example is to assume a linear mapping

$$\pi_{\boldsymbol{\theta}}\left(a|s\right) = \textit{Soft}_{\max}\left(\boldsymbol{\Theta}\mathbf{x}\left(s\right)\right)$$

Recall that Softmax is defined as

$$\textit{Soft}_{\max}\left(\mathbf{z}\right) = \frac{\exp\left\{\mathbf{z}\right\}}{\int_{a} \exp\left\{\left[\mathbf{z}\right]_{a}\right\}}$$

? *What are the dimensions of $\boldsymbol{\Theta}$?*

So, we have

$$\int_{a} \pi_{\boldsymbol{\theta}}\left(a|s\right) = \sum_{a}\left[\textit{Soft}_{\max}\left(\boldsymbol{\Theta}\mathbf{x}\left(s\right)\right)\right]_{a} = 1$$

Example: *Gaussian*

For continuous actions, we can use Gaussian distribution: a linear model is

$$\pi_{\boldsymbol{\theta}}(a|s) \equiv \mathcal{N}\left(\boldsymbol{\theta}^{\mathsf{T}}\mathbf{x}(s), \sigma^2\right)$$
$$= \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left\{-\frac{\left(a - \boldsymbol{\theta}^{\mathsf{T}}\mathbf{x}(s)\right)^2}{2\sigma^2}\right\}$$

We may train by either only learning $\boldsymbol{\theta}$ or both $\boldsymbol{\theta}$ and σ^2

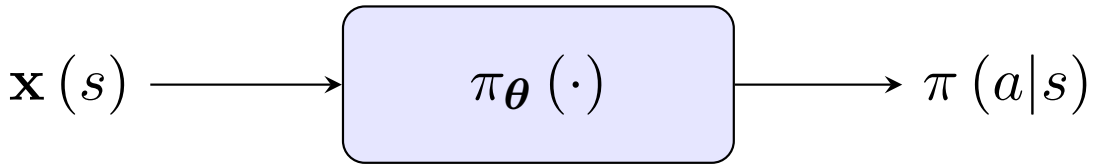
Stochastic Policy

Here, each time we see a state s, we sample an action

$$a \sim \pi_{\boldsymbol{\theta}}(a|s) \equiv \mathcal{N}\left(\boldsymbol{\theta}^{\mathsf{T}}\mathbf{x}(s), \sigma^2\right)$$

Training Policy Network

Let's now train the policy network: *assume a general network as*



? *How can we train it? What should be the loss?*

Ultimate Goal

We want ro have a policy that maximizes value at all states, i.e.,

$$\boldsymbol{\theta}^{\star} = \underset{\boldsymbol{\theta}}{\operatorname{argmax}}\, v_{\pi_{\boldsymbol{\theta}}}(s) = \underset{\boldsymbol{\theta}}{\operatorname{argmin}} -v_{\pi_{\boldsymbol{\theta}}}(s)$$

for all states $s \in \mathbb{S}$

Training Policy Network

+ *But that is weird! We have so many states! For which one we should do it?!*
– That's right! We should find a way around it

This naive training reduces to a multi-objective optimization

$$\boldsymbol{\theta}^{\star} = \underset{\boldsymbol{\theta}}{\operatorname{argmin}} -v_{\pi_{\boldsymbol{\theta}}}(s)$$

with the number of objectives being as much as the number of states!

Say we have N states: we need to have simultaneously

$$\boldsymbol{\theta}^{\star} = \underset{\boldsymbol{\theta}}{\operatorname{argmin}} -v_{\pi_{\boldsymbol{\theta}}}\left(s^1\right) \qquad \ldots \qquad \boldsymbol{\theta}^{\star} = \underset{\boldsymbol{\theta}}{\operatorname{argmin}} -v_{\pi_{\boldsymbol{\theta}}}\left(s^N\right)$$

which is not necessarily possible!

Finding Loss

A classical remedy to such multi-objective optimization is to scalarize

$$\boldsymbol{\theta}^{\star} = \underset{\boldsymbol{\theta}}{\operatorname{argmin}} -p\left(s^1\right) v_{\pi_{\boldsymbol{\theta}}}\left(s^1\right) - \ldots - p\left(s^N\right) v_{\pi_{\boldsymbol{\theta}}}\left(s^N\right)$$

Or better to say: to minimize the average return over all states, i.e.,

$$\begin{aligned} \mathcal{J}\left(\pi_{\boldsymbol{\theta}}\right) &= \int_{s} v_{\pi_{\boldsymbol{\theta}}}(s)\, p(s) \\ &= \mathbb{E}_{S \sim p}\left\{v_{\pi_{\boldsymbol{\theta}}}(S)\right\} \end{aligned}$$

+ *OK! But what is $p(s)$?! Do we have it? Or shall we assume it?*
– *Neither and both ☺ Let's try a simple setting first*

Finding Loss

We have an episodic environment whose sample trajectory looks like

$$\tau : S_0, A_0 \xrightarrow{R_1} S_1, A_1 \xrightarrow{R_2} \cdots \xrightarrow{R_{T-1}} S_{T-1}, A_{T-1} \xrightarrow{R_T} S_T$$

We denote the whole trajectory by τ to keep our notation simple

---

Assume we have no discount; then, we could say that a sample return is

$$G_0 = \sum_{t=0}^{T-1} R_{t+1}$$

and that the value for sample state S_0

$$v_{\pi_{\boldsymbol{\theta}}}(S_0) = \mathbb{E}_{\pi_{\boldsymbol{\theta}}}\{G_0 | S_0\}$$

Finding Loss

Say we fix our starting state to $S_0 = s_0$: *we get a sample trajectory as*

$$\tau(s_0): s_0, A_0 \xrightarrow{R_1} S_1, A_1 \xrightarrow{R_2} \cdots \xrightarrow{R_{T-1}} S_{T-1}, A_{T-1} \xrightarrow{R_T} S_T$$

The value of the starting state s_0 is then given by

$$\begin{aligned} v_{\pi_{\boldsymbol{\theta}}}(s_0) &= \mathbb{E}_{\pi_{\boldsymbol{\theta}}}\{G_0|s_0\} \\ &= \int_{r_1,\ldots,r_T} \int_{s_1,\ldots,s_T} \int_{a_0,\ldots,a_{T-1}} \left(\sum_{t=0}^{T-1} r_{t+1}\right) \pi_{\boldsymbol{\theta}}(a_0|s_0)\, p(s_1, r_1|s_0, a_0) \\ &\qquad \ldots \pi_{\boldsymbol{\theta}}(a_{T-1}|s_{T-1})\, p(s_T, r_T|s_{T-1}, a_{T-1}) \\ &= \int_{\tau(s_0)} \left(\sum_{t=0}^{T-1} r_{t+1}\right) \prod_{t=0}^{T-1} \pi_{\boldsymbol{\theta}}(a_t|s_t)\, p(s_{t+1}, r_{t+1}|s_t, a_t) \end{aligned}$$

Finding Loss

$$\tau\left(s_0\right): s_0, A_0 \xrightarrow{R_1} S_1, A_1 \xrightarrow{R_2} \cdots \xrightarrow{R_{T-1}} S_{T-1}, A_{T-1} \xrightarrow{R_T} S_T$$

Let's define the return of this trajectory as

$$G\left(\tau\left(s_0\right)\right) = \sum_{t=0}^{T-1} R_{t+1}$$

which is a random variable with outcomes

$$\mathrm{g}\left(\tau\left(s_0\right)\right) = \sum_{t=0}^{T-1} r_{t+1}$$

We can now write

$$v_{\pi_{\boldsymbol{\theta}}}\left(s_0\right) = \int_{\tau(s_0)} \mathrm{g}\left(\tau\left(s_0\right)\right) \prod_{t=0}^{T-1} \pi_{\boldsymbol{\theta}}\left(a_t | s_t\right) p\left(s_{t+1}, r_{t+1} | s_t, a_t\right)$$

Finding Loss

$$\tau\left(s_0\right): s_0, A_0 \xrightarrow{R_1} S_1, A_1 \xrightarrow{R_2} \cdots \xrightarrow{R_{T-1}} S_{T-1}, A_{T-1} \xrightarrow{R_T} S_T$$

Note that we could look at this term as an expectation

$$\begin{aligned} v_{\pi_{\boldsymbol{\theta}}}\left(s_0\right) &= \int_{\tau(s_0)} \mathrm{g}\left(\tau\left(s_0\right)\right) \prod_{t=0}^{T-1} \pi_{\boldsymbol{\theta}}\left(a_t|s_t\right) p\left(s_{t+1}, r_{t+1}|s_t, a_t\right) \\ &= \mathbb{E}_{\tau(s_0)\sim\prod_{t=0}^{T-1} \pi_{\boldsymbol{\theta}}(a_t|s_t)p(s_{t+1},r_{t+1}|s_t,a_t)} \left\{G\left(\tau\left(s_0\right)\right)\right\} \end{aligned}$$

Initial Conclusion

Distribution of $\tau\left(s_0\right)$ for a given s_0 which includes all next states is specified by policy and environment

Finding Loss

Now, let's assume a randomly chosen starting state S_0: *then, we have*

$$\tau : S_0, A_0 \xrightarrow{R_1} S_1, A_1 \xrightarrow{R_2} \cdots \xrightarrow{R_{T-1}} S_{T-1}, A_{T-1} \xrightarrow{R_T} S_T$$

We choose it with some distribution $p(s_0)$; thus, we have

$$\begin{aligned}
\mathcal{J}(\pi_{\boldsymbol{\theta}}) &= \mathbb{E}_{S_0 \sim p}\left\{v_{\pi_{\boldsymbol{\theta}}}(S_0)\right\} \\
&= \int_{s_0} \int_{\tau(s_0)} \mathrm{g}(\tau(s_0))\, p(s_0) \prod_{t=0}^{T-1} \pi_{\boldsymbol{\theta}}(a_t|s_t)\, p(s_{t+1}, r_{t+1}|s_t, a_t) \\
&= \int_{\tau} \mathrm{g}(\tau)\, p(s_0) \prod_{t=0}^{T-1} \pi_{\boldsymbol{\theta}}(a_t|s_t)\, p(s_{t+1}, r_{t+1}|s_t, a_t)
\end{aligned}$$

average over all possible trajectories

Finding Loss: *Estimable Form*

We can define the overall distribution of trajectory τ as

$$p_{\pi_{\boldsymbol{\theta}}}(\tau) = p(s_0) \prod_{t=0}^{T-1} \pi_{\boldsymbol{\theta}}(a_t|s_t)\, p(s_{t+1}, r_{t+1}|s_t, a_t)$$

Then we could compute the average return of the environment as

$$\begin{aligned} \mathcal{J}(\pi_{\boldsymbol{\theta}}) &= \int_{\tau} g(\tau)\, p_{\pi_{\boldsymbol{\theta}}}(\tau) \\ &= \mathbb{E}_{\tau \sim p_{\pi_{\boldsymbol{\theta}}}} \{G(\tau)\} \end{aligned}$$

return of trajectory

distribution of trajectory

Final Conclusion

Part of $p_{\pi_{\boldsymbol{\theta}}}$ is assumed and remaining is imposed policy and environment

Training Policy Network: *Gradient Descent*

We have the loss ready: *let's start training the policy network*

$$\boldsymbol{\theta}^{\star} = \underset{\boldsymbol{\theta}}{\operatorname{argmin}} -\mathcal{J}\left(\pi_{\boldsymbol{\theta}}\right) = \underset{\boldsymbol{\theta}}{\operatorname{argmin}} -\mathbb{E}_{\tau \sim p_{\pi_{\boldsymbol{\theta}}}}\left\{G\left(\tau\right)\right\}$$

We can use gradient descent: we consider learning rate α and update as

$$\begin{aligned}\boldsymbol{\theta} &\leftarrow \boldsymbol{\theta} - \alpha \nabla \left\{-\mathcal{J}\left(\pi_{\boldsymbol{\theta}}\right)\right\} \\ &\leftarrow \boldsymbol{\theta} + \alpha \nabla \mathcal{J}\left(\pi_{\boldsymbol{\theta}}\right)\end{aligned}$$

So, we need to compute $\nabla \mathcal{J}\left(\pi_{\boldsymbol{\theta}}\right)$ with respect to $\boldsymbol{\theta}$

Training Policy Network: *Gradient Descent*

We are using gradient descent (ascent)

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \alpha \nabla_{\boldsymbol{\theta}} \mathcal{J}\left(\pi_{\boldsymbol{\theta}}\right)$$

and we need the gradient: so, we write

$$\nabla_{\boldsymbol{\theta}} \mathcal{J}\left(\pi_{\boldsymbol{\theta}}\right) = \nabla_{\boldsymbol{\theta}} \int_{\tau} \mathrm{g}\left(\tau\right) p_{\pi_{\boldsymbol{\theta}}}\left(\tau\right) = \int_{\tau} \mathrm{g}\left(\tau\right) \nabla_{\boldsymbol{\theta}} p_{\pi_{\boldsymbol{\theta}}}\left(\tau\right)$$

$$= \int_{\tau} \mathrm{g}\left(\tau\right) \nabla_{\boldsymbol{\theta}} \left\{ \prod_{t=0}^{T-1} \pi_{\boldsymbol{\theta}}\left(a_t | s_t\right) p\left(s_{t+1}, r_{t+1} | s_t, a_t\right) p\left(s_0\right) \right\}$$

- \+ *It looks challenging!*
- – *Let's take a deeper look*

Training Policy Network: *Gradient Descent*

There is a trick that might help us in this respect

the so-called log-derivative trick

Log-Derivative Trick

For any positive function $f(\cdot) : \mathbb{R}^J \mapsto \mathbb{R}_+$ *we have by definition*

$$\nabla f(\boldsymbol{\theta}) = f(\boldsymbol{\theta}) \nabla \log f(\boldsymbol{\theta})$$

Let's apply the log-derivative trick to our problem

Training Policy Network: *Gradient Descent*

Applying the log-derivative trick to our problem, we have

$$\begin{aligned}
\nabla \mathcal{J}\left(\pi_{\boldsymbol{\theta}}\right) &= \int_{\tau} \mathrm{g}\left(\tau\right) \nabla p_{\pi_{\boldsymbol{\theta}}}\left(\tau\right) \\
&= \int_{\tau} \mathrm{g}\left(\tau\right) p_{\pi_{\boldsymbol{\theta}}}\left(\tau\right) \nabla \log p_{\boldsymbol{\theta}}\left(\tau\right) \\
&= \int_{\tau} \left[\mathrm{g}\left(\tau\right) \nabla \log p_{\pi_{\boldsymbol{\theta}}}\left(\tau\right)\right] p_{\pi_{\boldsymbol{\theta}}}\left(\tau\right) \; = \mathbb{E}_{\tau \sim p_{\pi_{\boldsymbol{\theta}}}} \left\{G\left(\tau\right) \nabla \log p_{\pi_{\boldsymbol{\theta}}}\left(\tau\right)\right\}
\end{aligned}$$

+ *Why should that be helpful?!*
- *Let's see how* $\log p_{\pi_{\boldsymbol{\theta}}}\left(\tau\right)$ *looks*

Training Policy Network: *Gradient Descent*

Consider one instant trajectory: *we have a particular outcome*

$$\tau : s_0, a_0 \xrightarrow{r_1} s_1, a_1 \xrightarrow{r_2} \cdots \xrightarrow{r_{T-1}} s_{T-1}, a_{T-1} \xrightarrow{r_T} s_T$$

Using the definition of $p_{\boldsymbol{\theta}}(\tau)$*, we can write*

$$\log p_{\pi_{\boldsymbol{\theta}}}(\tau) = \log \left\{ p(s_0) \prod_{t=0}^{T-1} \pi_{\boldsymbol{\theta}}(a_t|s_t)\, p(s_{t+1}, r_{t+1}|s_t, a_t) \right\}$$

$$= \underbrace{\log p(s_0)}_{\text{does not depend in } \boldsymbol{\theta}} + \sum_{t=0}^{T-1} \log \pi_{\boldsymbol{\theta}}(a_t|s_t) + \underbrace{\sum_{t=0}^{T-1} \log p(s_{t+1}, r_{t+1}|s_t, a_t)}_{\text{does not depend in } \boldsymbol{\theta}}$$

Training Policy Network: *Gradient Descent*

The gradient of $\log p_{\boldsymbol{\theta}}(\tau)$ is hence given by

$$\nabla \log p_{\pi_{\boldsymbol{\theta}}}(\tau) = \nabla \sum_{t=0}^{T-1} \log \pi_{\boldsymbol{\theta}}(a_t|s_t) = \sum_{t=0}^{T-1} \nabla \log \pi_{\boldsymbol{\theta}}(a_t|s_t)$$

If we think of a randomly sampled trajectory

$$\tau : S_0, A_0 \xrightarrow{R_1} S_1, A_1 \xrightarrow{R_2} \cdots \xrightarrow{R_{T-1}} S_{T-1}, A_{T-1} \xrightarrow{R_T} S_T$$

we can similarly write

$$\nabla \log p_{\pi_{\boldsymbol{\theta}}}(\tau) = \sum_{t=0}^{T-1} \nabla \log \pi_{\boldsymbol{\theta}}(A_t|S_t)$$

Training Policy Network: *Gradient Descent*

Back to our main problem: *we have a random trajectory*

$$\tau : S_0, A_0 \xrightarrow{R_1} S_1, A_1 \xrightarrow{R_2} \cdots \xrightarrow{R_{T-1}} S_{T-1}, A_{T-1} \xrightarrow{R_T} S_T$$

and want to find the gradient of loss; so, we can write

$$\begin{aligned}
\nabla \mathcal{J}(\pi_{\boldsymbol{\theta}}) &= \mathbb{E}_{\tau \sim p_{\pi_{\boldsymbol{\theta}}}} \left\{ G(\tau) \nabla \log p_{\pi_{\boldsymbol{\theta}}}(\tau) \right\} \\
&= \mathbb{E}_{\tau \sim p_{\pi_{\boldsymbol{\theta}}}} \left\{ G(\tau) \sum_{t=0}^{T-1} \nabla \log \pi_{\boldsymbol{\theta}}(A_t | S_t) \right\} \\
&= \mathbb{E}_{\tau \sim p_{\pi_{\boldsymbol{\theta}}}} \left\{ \left(\sum_{t=0}^{T-1} R_{t+1} \right) \sum_{t=0}^{T-1} \nabla \log \pi_{\boldsymbol{\theta}}(A_t | S_t) \right\}
\end{aligned}$$

We can estimate it via Monte-Carlo!

Training Policy Network: *SGD*

Say we set the weights of policy network to $\boldsymbol{\theta}$: *we sample* K *trajectories*

$$\tau : S_0, A_0 \xrightarrow{R_1} S_1, A_1 \xrightarrow{R_2} \cdots \xrightarrow{R_{T-1}} S_{T-1}, A_{T-1} \xrightarrow{R_T} S_T$$

from the environment using policy $\pi_{\boldsymbol{\theta}}$*, and then estimate the gradient as*

$$\hat{\nabla}\mathcal{J}\left(\pi_{\boldsymbol{\theta}}\right) = \frac{1}{K}\sum_{k=1}^{K} G\left(\tau_k\right) \sum_{t=0}^{T-1} \nabla \log \pi_{\boldsymbol{\theta}}\left(A_t\left[k\right] | S_t\left[k\right]\right)$$

We can then use gradient descent to update $\boldsymbol{\theta}$ *as*

$$\begin{aligned}
\boldsymbol{\theta} &\leftarrow \boldsymbol{\theta} + \alpha\hat{\nabla}\mathcal{J}\left(\pi_{\boldsymbol{\theta}}\right) \\
&\leftarrow \boldsymbol{\theta} + \frac{\alpha}{K}\sum_{k=1}^{K} G\left(\tau_k\right) \sum_{t=0}^{T-1} \nabla \log \pi_{\boldsymbol{\theta}}\left(A_t\left[k\right] | S_t\left[k\right]\right)
\end{aligned}$$

Training Policy Network: *SGD*

+ *Isn't that again too slow?! We should wait for a single update!*
- Sure! We can go for SGD

Using SGD, we could take a single sample gradient

$$\hat{\nabla}\mathcal{J}\left(\pi_{\boldsymbol{\theta}}\right) = G\left(\tau\right)\sum_{t=0}^{T-1}\nabla\log\pi_{\boldsymbol{\theta}}\left(A_t|S_t\right)$$

and then update the policy network as

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \alpha G\left(\tau\right)\sum_{t=0}^{T-1}\nabla\log\pi_{\boldsymbol{\theta}}\left(A_t|S_t\right)$$

First Policy Gradient Algorithm

`PG_v1()`:

1: *Initialize with* $\boldsymbol{\theta}$ *and learning rate* α
2: ***for*** *episode* $= 1 : K$ ***do***
3: *Sample a trajectory with policy* $\pi_{\boldsymbol{\theta}}$

$$\tau : S_0, A_0 \xrightarrow{R_1} S_1, A_1 \xrightarrow{R_2} \cdots \xrightarrow{R_{T-1}} S_{T-1}, A_{T-1} \xrightarrow{R_T} S_T$$

4: *Compute return* $G(\tau)$
5: ***for*** $t = 0 : T - 1$ ***do***
6: *Update policy network* $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \alpha G(\tau) \nabla \log \pi_{\boldsymbol{\theta}}(A_t | S_t)$
7: ***end for***
8: ***end for***

With a bit of modification *this reduces to REINFORCE algorithm proposed by Ronald J. Williams in 1992*

REINFORCE: *First Official Algorithm*

`REINFORCE():`

1: *Initialize with* $\boldsymbol{\theta}$ *and learning rate* α
2: ***for*** *episode* $= 1 : K$ ***do***
3: *Sample a trajectory with policy* $\pi_{\boldsymbol{\theta}}$

$$\tau : S_0, A_0 \xrightarrow{R_1} S_1, A_1 \xrightarrow{R_2} \cdots \xrightarrow{R_{T-1}} S_{T-1}, A_{T-1} \xrightarrow{R_T} S_T$$

4: ***for*** $t = 0 : T - 1$ ***do***
5: *Update policy network* $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \alpha G_t \nabla \log \pi_{\boldsymbol{\theta}} (A_t | S_t)$
6: ***end for***
7: ***end for***

+ *But we are now computing a different gradient? Why should it work?!*
– This is because of the *Policy Gradient Theorem* *which says that we should update proportional to* $\nabla \log \pi_{\boldsymbol{\theta}} (A_t | S_t)$

SGD: *General Setting*

Let's have a more generic analysis: *assume we start at a random state S_0 that is chosen according to*

$$S_0 \sim p(s_0)$$

We start acting via the policy $\pi_{\boldsymbol{\theta}}$ and transit to a new state

$$S_0, A_0 \xrightarrow{R_1} S_1$$

We could then say that the average value of the policy is

$$\mathcal{J}(\pi_{\boldsymbol{\theta}}) = \mathbb{E}_{S_0 \sim p}\{v_{\pi_{\boldsymbol{\theta}}}(S_0)\}$$

We need the gradient of this value against $\boldsymbol{\theta}$ to train the policy network

SGD: *General Setting*

We can open up the loss expression

$$\mathcal{J}\left(\pi_{\boldsymbol{\theta}}\right) = \int_{s_0} v_{\pi_{\boldsymbol{\theta}}}\left(s_0\right) p\left(s_0\right)$$

and write the gradient as

$$\begin{aligned}\nabla \mathcal{J}\left(\pi_{\boldsymbol{\theta}}\right) &= \nabla \int_{s} v_{\pi_{\boldsymbol{\theta}}}\left(s_0\right) p\left(s_0\right) \\ &= \int_{s} \nabla v_{\pi_{\boldsymbol{\theta}}}\left(s_0\right) p\left(s_0\right)\end{aligned}$$

Let's compute $\nabla v_{\pi_{\boldsymbol{\theta}}}\left(s_0\right)$

SGD: *General Setting*

We can use the marginalization rule to expand $v_{\pi_{\boldsymbol{\theta}}}(s_0)$

$$v_{\pi_{\boldsymbol{\theta}}}(s_0) = \int_{a_0} q_{\pi_{\boldsymbol{\theta}}}(s_0, a_0)\, \pi_{\boldsymbol{\theta}}(a_0|s_0)$$

So the gradient $\nabla v_{\pi_{\boldsymbol{\theta}}}(s_0)$ *is computed using chain rule as*

$$\begin{aligned}\nabla v_{\pi_{\boldsymbol{\theta}}}(s_0) &= \nabla \int_{a_0} q_{\pi_{\boldsymbol{\theta}}}(s_0, a_0)\, \pi_{\boldsymbol{\theta}}(a_0|s_0) \\ &= \int_{a_0} \nabla q_{\pi_{\boldsymbol{\theta}}}(s_0, a_0)\, \pi_{\boldsymbol{\theta}}(a_0|s_0) + \int_{a_0} q_{\pi_{\boldsymbol{\theta}}}(s_0, a_0)\, \nabla \pi_{\boldsymbol{\theta}}(a_0|s_0)\end{aligned}$$

Let's compute $\nabla q_{\pi_{\boldsymbol{\theta}}}(s_0, a_0)$ *next*

SGD: *General Setting*

We can use Bellman equation to expand $q_{\pi_{\boldsymbol{\theta}}}(s_0, a_0)$ *as*

$$q_{\pi_{\boldsymbol{\theta}}}(s_0, a_0) = \mathcal{R}(s_0, a_0) + \gamma \int_{s_1} v_{\pi_{\boldsymbol{\theta}}}(s_1)\, p(s_1|s_0, a_0)$$

So the gradient reads

$$\begin{aligned}
\nabla q_{\pi_{\boldsymbol{\theta}}}(s_0, a_0) &= \nabla \left\{ \mathcal{R}(s_0, a_0) + \gamma \int_{s_1} v_{\pi_{\boldsymbol{\theta}}}(s_1)\, p(s_1|s_0, a_0) \right\} \\
&= \underbrace{\nabla \mathcal{R}(s_0, a_0)}_{0} + \gamma \int_{s_1} \nabla v_{\pi_{\boldsymbol{\theta}}}(s_1)\, p(s_1|s_0, a_0) \\
&= \gamma \int_{s_1} \nabla v_{\pi_{\boldsymbol{\theta}}}(s_1)\, p(s_1|s_0, a_0)
\end{aligned}$$

SGD: *General Setting*

Now, let's put back all gradients gradually towards beginning of computation

$$\begin{aligned}\nabla \mathcal{J}(\pi_{\boldsymbol{\theta}}) &= \int_{s_0} \nabla v_{\pi_{\boldsymbol{\theta}}}(s_0)\, p(s_0) \\ &= \int_{s_1} \nabla v_{\pi_{\boldsymbol{\theta}}}(s_1)\, p_{\pi_{\boldsymbol{\theta}}}(s_1) + \int_{s_0}\int_{a_0} q_{\pi_{\boldsymbol{\theta}}}(s_0, a_0)\, \nabla \pi_{\boldsymbol{\theta}}(a_0|s_0)\, p(s_0)\end{aligned}$$

where we define the marginal distribution of s_1 as

$$p_{\pi_{\boldsymbol{\theta}}}(s_1) = \int_{s_0}\int_{a_0} p(s_1|s_0, a_0)\, \pi_{\boldsymbol{\theta}}(a_0|s_0)\, p(s_0)$$

SGD: *General Setting*

Since $p_{\pi_{\boldsymbol{\theta}}}(s_1)$ and $p(s_0)$ are distributions, we have

$$\int_{s_1} p_{\pi_{\boldsymbol{\theta}}}(s_1) = \int_{s_0} p(s_0) = 1$$

So, we could modify our final expression as

$$\begin{aligned}
\nabla \mathcal{J}(\pi_{\boldsymbol{\theta}}) = & \int_{s_1} \nabla v_{\pi_{\boldsymbol{\theta}}}(s_1)\, p_{\pi_{\boldsymbol{\theta}}}(s_1) \int_{s_0} p(s_0) \\
& + \int_{s_0}\int_{a_0} q_{\pi_{\boldsymbol{\theta}}}(s_0, a_0)\, \nabla \pi_{\boldsymbol{\theta}}(a_0|s_0)\, p(s_0) \int_{s_1} p_{\pi_{\boldsymbol{\theta}}}(s_1) \\
& = \int_{s_1}\int_{s_0} p_{\pi_{\boldsymbol{\theta}}}(s_1)\, p(s_0) \left[\nabla v_{\pi_{\boldsymbol{\theta}}}(s_1) + \int_{a_0} q_{\pi_{\boldsymbol{\theta}}}(s_0, a_0)\, \nabla \pi_{\boldsymbol{\theta}}(a_0|s_0) \right]
\end{aligned}$$

SGD: *General Setting*

If we keep on progressing in the trajectory as $t \to \infty$, we will see

$$\nabla \mathcal{J}(\pi_{\boldsymbol{\theta}}) = \int_s d_{\pi_{\boldsymbol{\theta}}}(s) \int_a q_{\pi_{\boldsymbol{\theta}}}(s,a) \nabla \pi_{\boldsymbol{\theta}}(a|s)$$

for some distribution $d_{\pi_{\boldsymbol{\theta}}}(s)$ that is the average marginal distribution of states under policy $\pi_{\boldsymbol{\theta}}$, i.e.,

$$\int_s d_{\pi_{\boldsymbol{\theta}}}(s) = 1$$

Finally, using the log-derivative trick we have

$$\nabla \mathcal{J}(\pi_{\boldsymbol{\theta}}) = \int_s d_{\pi_{\boldsymbol{\theta}}}(s) \int_a q_{\pi_{\boldsymbol{\theta}}}(s,a) \pi_{\boldsymbol{\theta}}(a|s) \nabla \log \pi_{\boldsymbol{\theta}}(a|s)$$

Policy Gradient Theorem

This can be equivalently written as

$$\nabla \mathcal{J}(\pi_{\boldsymbol{\theta}}) = \int_s \int_a d_{\pi_{\boldsymbol{\theta}}}(s)\, \pi_{\boldsymbol{\theta}}(a|s) q_{\pi_{\boldsymbol{\theta}}}(s,a)\, \nabla \log \pi_{\boldsymbol{\theta}}(a|s)$$
$$= \mathbb{E}_{S\sim d_{\pi_{\boldsymbol{\theta}}}, A|S\sim\pi_{\boldsymbol{\theta}}} \{q_{\pi_{\boldsymbol{\theta}}}(S,A)\, \nabla \log \pi_{\boldsymbol{\theta}}(A|S)\}$$

which concludes the policy gradient theorem proved by Sutton et al. in 1992

Policy Gradient Theorem

For a policy network with non-zero probabilities, the gradient of the average trajectory return is always given by

$$\nabla \mathcal{J}(\pi_{\boldsymbol{\theta}}) = \mathbb{E}_{S\sim d_{\pi_{\boldsymbol{\theta}}}, A|S\sim\pi_{\boldsymbol{\theta}}} \{q_{\pi_{\boldsymbol{\theta}}}(S,A)\, \nabla \log \pi_{\boldsymbol{\theta}}(A|S)\}$$